

# **CISP 400**

## Object Oriented Programming in C++

Dr. Fowler



F O L S O M   L A K E   C O L L E G E  
EL DORADO CENTER | RANCHO CORDOVA CENTER

# CODE LIFESPAN

|<

< PREV

RANDOM

NEXT >

>|

IT TOOK SOME EXTRA  
WORK TO BUILD, BUT  
NOW WE'LL BE ABLE  
TO USE IT FOR ALL OUR  
FUTURE PROJECTS.



HOW TO ENSURE YOUR  
CODE IS NEVER REUSED

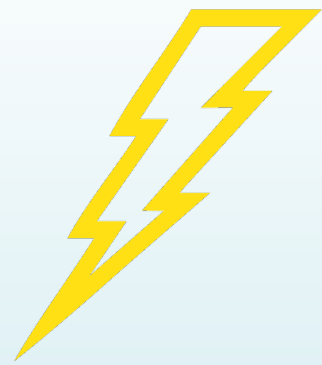
LET'S NOT OVERTHINK IT;  
IF THIS CODE IS STILL IN  
USE *THAT* FAR IN THE  
FUTURE, WE'LL HAVE  
BIGGER PROBLEMS.



HOW TO ENSURE YOUR  
CODE LIVES FOREVER

A large, stylized yellow lightning bolt graphic is positioned diagonally across the center of the slide. It has a jagged, zig-zag shape with a sharp point at the bottom left and a flat top right.

# Lightning Review



# Lightning Review Questions

**Great time to get Peer Review Partners .....**

- What kind of behaviors would your classes need to exhibit to resemble classes normally found in C++ (like strings)?
- What other questions to you have for me?

# Theme for Today

“There’s more to Objects than you imagined.”

# Chapter 11

## More OOP



# Ch 11 - More OOP

11.1 The this pointer and Constant member Functions

11.2 Static Members

11.3 Friends of Classes

11.4 Memberwise Assignment

11.5 Copy Constructors

11.6 Operator Overloading

11.7 Rvalue References and Move Operations

11.8 Function Objects and Lambda Expressions

11.9 Type Conversion Operators

11.10 Convergent Constructors

11.11 Aggregation and Composition

11.12 Inheritance

11.13 Protected Members and Class Access

11.14 Constructors, Destructors, and Inheritance

11.15 Overriding Base Class Functions



# Ch 11 - More OOP

## **11.1 The this pointer and Constant member Functions**

## **11.2 Static Members**

## **11.3 Friends of Classes**

## 11.4 Memberwise Assignment

## 11.5 Copy Constructors

## 11.6 Operator Overloading

## 11.7 Rvalue References and Move Operations

## 11.8 Function Objects and Lambda Expressions

## 11.9 Type Conversion Operators

## 11.10 Convergent Constructors

## 11.11 Aggregation and Composition

## 11.12 Inheritance

## 11.13 Protected Members and Class Access

## 11.14 Constructors, Destructors, and Inheritance

## 11.15 Overriding Base Class Functions

# 7.6 Constructors

# Constructors

- Run when the class is instantiated.
- Same name as the class.
- No return type at all (not even void).

- Does this mean classes which we have not created a constructor, do not have constructors?

## **Constructor Use Case**

Set up connections between objects (DB, network, etc)

Initialize data.

Serve as an interface.

## Constructors as an interface

We usually have data we want to initialize with when we instantiate our constructor.

What if we don't have all the data? Does that mean we can't instantiate?

# Overloading Constructors



- Computer always makes a default constructor - unless you write ANY other constructor.
- Meaning: you will not have a default constructor if you've overloaded it.
- Meaning: you cannot create an object when you do not pass any values upon instantiation. Refuses to compile.
- Typical default constructor is not passed any arguments.
- However, we can have a default constructor with arguments if we set the parameters with default values.
- We can only have one default constructor. We can have unlimited number of overloaded constructors.



# Bottom Line

- If you are adding a constructor which accepts arguments, you will also need to create a default constructor.
- A default constructor does not accept any arguments, unless you make default parameters available.

```

1 // 7-6.cpp -- Overloading constructors
2 #include <iostream>
3 #include <iomanip>
4 using namespace std;
5
6 class Sale {
7 private:
8     double taxRate;
9 public:
10    Sale(double rate)           // constructor with 1 parameter
11    { taxRate = rate;
12    }
13
14    Sale()                     // default constructor
15    { taxRate = 0.0;
16    }
17
18    double calcSaleTotal(double cost) {
19        double total = cost + cost * taxRate;
20        return total;
21    }
22 };
23
24 int main()
25 {
26     Sale cashier1(.06);
27     Sale cashier2;
28
29     // format the output
30     cout << fixed << showpoint << setprecision(2);
31
32     // Get and display the total sale price for two $24.95 sales
33     cout << "With a 0.06 sale tax rate, the total \n";
34     cout << "of the $24.95 sale is $";
35     cout << cashier1.calcSaleTotal(24.95) << endl;
36
37     cout << "\nOn a tax-exempt sale, the total\n";
38     cout << "of the $24.95 sale is $";
39     cout << cashier2.calcSaleTotal(24.95) << endl;
40     return 0;
41 }

```

```

1 // 7-6.cpp -- Overloading constructors
2 #include <iostream>
3 #include <iomanip>
4 using namespace std;
5
6 class Sale {
7 private:
8     double taxRate;
9 public:
10    Sale(double rate)           // constructor with 1 parameter
11    { taxRate = rate;
12    }
13
14    Sale()                     // default constructor
15    { taxRate = 0.0;
16    }
17
18    double calcSaleTotal(double cost) {
19        double total = cost + cost * taxRate;
20        return total;
21    }
22 };
23
24 int main()
25 {
26     Sale cashier1(.06);
27     Sale cashier2;
28
29     // format the output
30     cout << fixed << showpoint << setprecision(2);
31
32     // Get and display the total sale price for two $24.95 sales
33     cout << "With a 0.06 sale tax rate, the total \n";
34     cout << "of the $24.95 sale is $";
35     cout << cashier1.calcSaleTotal(24.95) << endl;
36
37     cout << "\nOn a tax-exempt sale, the total\n";
38     cout << "of the $24.95 sale is $";
39     cout << cashier2.calcSaleTotal(24.95) << endl;
40     return 0;
41 }

```

With a 0.06 sale tax rate, the total of the \$24.95 sale is \$26.45

On a tax-exempt sale, the total of the \$24.95 sale is 24.95

...Program finished with exit code 0  
Press ENTER to exit console.

Constructors, functions  
and methods, Oh my!

- Distinguishing between various constructors and methods can be confusing.

```
Class ABC {  
    };
```

```
int main() {  
    ABC x;          // Default - works fine  
    ABC x();        // Error - thinks it's a method  
    ABC x(10)       // Fine - overloaded default
```

# Exercise

C++ Debugging



# Exercise Goals

- Debugging Practice



# Debug Exercise

- Correct the following code.
- We'll spend about 5 minutes on this, then we'll go over it.
- You do not need to turn this in.

```
1  // debug10-2.cpp -- find the errors
2
3  class syntaxErrors1
4  {
5  public:
6      syntaxErrors();
7      void setData(double, double);
8      int mult();
9  private:
10     int one;
11     double two;
12 }
```

# Exercise

C++ Debugging

# Exercise Goals

- Debugging Practice

# Debug Exercise

- Correct the following code.
- We'll spend about 5 minutes on this, then we'll go over it.
- You do not need to turn this in.

```
1 // debug10-3.cpp -- find the errors
2 // Ma18 p.724
3
4 class syntaxErrors2
5 {
6 ▼ public:
7     bool canGraduate();
8     void print() const;
9     void syntaxErrors2(int = 0,
10                        |         |         |         |         |         double = 0);
11
12 ▼ private;
13     int ID;
14     double gpa;
15 };
```

# 7.7 Destructors



```

1 // stoplight2.cpp
2 #include <iostream>
3 #include <string>
4 using namespace std;
5
6 class StopLight {
7     private:
8         bool isPowered = false;
9         int circuitBreaker = 3;
10        int delay = 0;
11
12    public:
13        StopLight();
14        ~StopLight();
15        void PowerOn();
16        void PowerOff();
17        void Cycle();
18 };
19 StopLight::StopLight() {
20     cout << "Powering on ..." << endl;
21     delay = 100000000;
22 }
23 void StopLight::PowerOn() {
24     isPowered = true;
25 }
26 void StopLight::PowerOff() {
27     isPowered = false;

```

```

28 }
29 void StopLight::Cycle() {
30     int i = 0;
31     while (isPowered)
32     {
33         cout << "Green" << endl;
34         for (int i = 0; i < delay; i++)
35             { ; }
36         cout << "Yellow" << endl;
37         for (int i = 0; i < delay; i++)
38             { ; }
39         cout << "Red" << endl;
40         for (int i = 0; i < delay; i++)
41             ;
42         if (i < circuitBreaker)
43             i++;
44         else
45             PowerOff();
46     }
47 }
48 StopLight::~~StopLight() {
49     cout << "Powering Off ..." << endl;
50 }
51
52 int main() {
53     StopLight bidwell;
54     bidwell.PowerOn();
55     bidwell.Cycle();
56 }

```





- Constructors!
- Destructors!

# 11.1 The `this` pointer and Constant member Functions

# The Problem

You have an object, myWidget.

You need to make a copy of it to yourWidget.

How can we do that?

# Problem

Our object has a class variable.

It also has a method variable with the same name.

Is this a problem?

```

1 // 11-1x.cpp -- this demo problem
2 #include <iostream>
3 using namespace std;
4
5 class Example{
6 private:
7     int x;
8 public:
9     Example(int a){ x = a;}
10    void setValue(int);
11    void printAddressAndValue();
12 };
13
14 void Example::setValue(int a)
15 { x = a; }
16
17 void Example::printAddressAndValue(){
18     int x = 50;
19     cout << "Object at address: " << &x << " has "
20         << "x value of: " << x << endl;
21 }
22
23 int main() {
24     Example obj1(10), obj2(20);
25
26     obj1.printAddressAndValue();
27     obj2.printAddressAndValue();
28
29     return 0;
30 }
31

```

gcc version 4.6.3

```

>
Object at address: 0x7ffead507c7c has x value of: 50
Object at address: 0x7ffead507c7c has x value of: 50
> █

```

- Referring to parts of an object.
- Referring to the entire object.

## **this pointer**

How about we create an object pointer variable which holds the address of itself? We can use it for the object to refer to itself (kinda like the royal “we”).



```

1 // 11-1.cpp -- this demo
2 #include <iostream>
3 using namespace std;
4
5 class Example{
6 private:
7     int x;
8 public:
9     Example(int a){ x = a;}
10    void setValue(int);
11    void printAddressAndValue();
12 };
13
14 void Example::setValue(int a)
15 { x = a; }
16
17 void Example::printAddressAndValue(){
18     int x = 50;
19     cout << "Object at address: " << this << " has "
20     << "x value of: " << (*this).x << endl;
21 }
22
23 int main() {
24     Example obj1(10), obj2(20);
25
26     obj1.printAddressAndValue();
27     obj2.printAddressAndValue();
28
29     return 0;
30 }

```

```

Object at address: 0x7ffeeb93baf8 has x value of: 10
Object at address: 0x7ffeeb93baf0 has x value of: 20
[Finished in 0.2s]

```





- Constructors!
- Destructors!
- `this` pointer

# What's Going On?

# Use Case

- We want to create a Rec class.
- We then want to add 2 Rec objects together.
- Let's see what happens.

# First Example

- Let's build an `Add()` function

```

1 //Op0n00P.cpp
2 // Operations on objects
3 #include <iostream>
4 using namespace std;
5
6 class Rec {
7     int x, y;
8 public:
9     Rec(int xx = 1, int yy = 1) {
10         x = xx; y = yy; }
11     void Print() {cout << x << " by " << y << endl;}
12     int GetX() {return x;}
13     int GetY() {return y;}
14 };
15
16 Rec Add (Rec, Rec);
17
18 int main() {
19     Rec a, b( 5, 5);
20     a.Print();
21     b.Print();
22
23     Rec c = Add(a, b);
24     c.Print();
25
26     return 0;
27 }
28
29 Rec Add (Rec lhs, Rec rhs) {
30     Rec tmp(lhs.GetX() + rhs.GetX(), lhs.GetY() + rhs.GetY());
31     return tmp;
32 }

```

# Second Example

- Let's make `Add()` a method instead.

```
1 //Op0n00P2.cpp
2 // Operations on objects
3 #include <iostream>
4 using namespace std;
5
6 class Rec {
7     int x, y;
8 public:
9     Rec(int xx = 1, int yy = 1) {
10         x = xx; y = yy; }
11     void Print() {cout << x << " by " << y << endl;}
12 // Rec Add(Rec, Rec);      not needed!
13     Rec Add(Rec);          // call is a.add(b);
14 };
15
16 Rec Rec::Add (Rec rhs){
17     Rec tmp(x + rhs.x, this->y + rhs.y);
18     return tmp;
19 }
20
21 int main() {
22     Rec a, b( 5, 5);
23     a.Print();
24     b.Print();
25
26     Rec c = a.Add(b);
27     c.Print();
28
29     return 0;
30 }
```



# What this means ...

- Adding primitive operations (+, -, /, \*) to our objects makes them more useful.
- Using methods instead of functions can be advantageous when we need to manipulate objects because we have an easier time accessing private data.
- We can also use our constructors inside our methods to save even more typing.



# 11.2 Static Members

# Problem

How can we create a shared variable across all our class instances?

```

1 // static.cpp -- static demo
2 #include <iostream>
3 using namespace std;
4
5 class Example{
6 private:
7     int x;
8     static int count;        // static variable
9 public:
10    Example(int a){ x = a; count++;}
11    static int getCount()    // static member function
12    { return count; }
13 };
14
15 int Example::count = 0;    // initialize
16
17 int main() {
18     Example obj1(10),
19             obj2(20);
20
21     cout << "Number of instances: " << obj1.getCount() << endl;
22
23     return 0;
24 }

```

Number of instances: 2  
[Finished in 0.2s]

```

1 // static.cpp -- static demo
2 #include <iostream>
3 using namespace std;
4
5 class Example{
6 private:
7     int x;
8     static int count;      // static variable
9 public:
10    Example(int a){ x = a; count++;}
11    static int getCount()  // static member function
12        { return count; }
13 };
14
15 int Example::count = 0;    // initialize
16
17 int main() {
18     cout << "Callable even w/o an object! count: "
19         << Example::getCount() << endl;
20
21     Example obj1(10),
22             obj2(20);
23
24     cout << "Number of instances: " << obj1.getCount() << endl;
25
26     return 0;
27 }

```

```

Callable even w/o an object! count: 0
Number of instances: 2

```

```

...Program finished with exit code 0
Press ENTER to exit console.

```

# Static Members

- Callable even if no instance of object!
- Only access static data member, other static members and functions outside the class.

## **Creational Patterns**

1. Abstract Factory
2. Builder
3. Factory Method
4. Prototype
5. Singleton

## **Structural Patterns**

1. Adapter
2. Bridge
3. Composite
4. Decorator
5. Façade
6. Flyweight
7. Proxy

## **Behavioral Patterns**

1. Chain of Responsibility
2. Command
3. Interpreter
4. Iterator
5. Mediator
6. Memento
7. Observer
8. State
9. Strategy
10. Template Method
11. Visitor

**Gang of Four (GoF) Design Patterns**

## Creational Patterns

1. Abstract Factory
2. Builder
3. Factory Method
4. Prototype
5. Singleton

## Structural Patterns

1. Adapter
2. Bridge
3. Composite
4. Decorator
5. Façade
6. Flyweight
7. Proxy

## Behavioral Patterns

1. Chain of Responsibility
2. Command
3. Interpreter
4. Iterator
5. Mediator
6. Memento
7. Observer
8. State
9. Strategy
10. Template Method
11. Visitor

Gang of Four (GoF) Design Patterns



```

1  // sing.cpp -- singleton with static variable
2  #include <iostream>
3  #include <ctime>
4  #include <cstdlib>
5  using namespace std;
6
7  class D6{
8  public:
9      int x;
10     static bool isInitialized;           // static variable
11 public:
12     D6(int a = 0){
13         if (!(isInitialized)) {
14             srand(time(NULL));
15             isInitialized = true;
16             cout << "Init code run\n";
17         }
18         x = rand() % 6 + 1;
19     }
20 };
21
22 bool D6::isInitialized = false;         // initialize
23
24 int main() {
25     D6 aD6, bD6;
26     cout << aD6.x << endl;
27     cout << bD6.x << endl;
28
29     return 0;
30 }
31

```

```

Init code run
6
4

```



```

1 //sing2.cpp -- singleton with heap variable
2 #include <iostream>
3 #include <ctime>
4 #include <cstdlib>
5 using namespace std;
6
7 class D6{
8 public:
9     int x;
10    bool* pInit;        // static variable
11 public:
12    D6(bool* ptr){
13        pInit = ptr;
14        if (!(*pInit)) {
15            srand(time(NULL));
16            *pInit = true;
17            cout << "Init code run\n";
18        }
19        x = rand() % 6 + 1;
20    }
21 };
22
23 class D20{
24 public:
25     int x;
26     bool* pInit;        // static variable
27 public:
28    D20(bool* ptr){
29        pInit = ptr;
30        if (!(*pInit)) {
31            srand(time(NULL));
32            *pInit = true;
33            cout << "Init code run\n";
34        }
35        x = rand() % 6 + 1;
36    }
37 };
38 bool* isInitialized;    // initialize
39
40 int main() {
41     isInitialized = new bool{false};
42     D6 aD6(isInitialized), bD6(isInitialized);
43     D20 aD20(isInitialized), bD20(isInitialized);
44     cout << aD6.x << endl;
45     cout << bD6.x << endl;
46     cout << aD20.x << endl;
47     cout << bD20.x << endl;
48
49     return 0;

```

```

./main
Init code run
6
1
6
5

```

# Programming Exercise

Activity

# Exercise Goals

- Increase Topic Mastery
- Construct Knowledge
- Give you data to evaluate how well you are learning.

# Programming Exercise

- Create a class Point2D that represents a point in a 2D plane with x and y coordinates. Overload the << operator to print the coordinates of the point in the format (x, y), and overload the >> operator to input the x and y coordinates from the user. Use friend functions for both operator overloads.\ul>- The << operator should display the point's coordinates in the format: (x, y).
- The >> operator should allow the user to input the x and y coordinates of the point.
- The overloaded operators must be implemented as friend functions.



What's this section  
about?

# Ch 11 - More OOP

## **11.1 The this pointer and Constant member Functions**

## **11.2 Static Members**

11.3 Friends of Classes

11.4 Memberwise Assignment

11.5 Copy Constructors

11.6 Operator Overloading

11.7 Rvalue References and Move Operations

11.8 Function Objects and Lambda Expressions

11.9 Type Conversion Operators

11.10 Convergent Constructors

11.11 Aggregation and Composition

11.12 Inheritance

11.13 Protected Members and Class Access

11.14 Constructors, Destructors, and Inheritance

11.15 Overriding Base Class Functions



NEXT TIME



# Chapter 11 - More About Classes and OOP

- 11.1 The this pointer and constant member functions
- 11.2 Static members
- **11.3 Friends of Classes**
- 11.4 Memberwise Assignment
- 11.5 Copy Constructors
- **11.6 Operator Overloading**
- 11.7 Rvalue References and Move Operations
- 11.8 Function Objects and Lambda Expressions
- 11.9 Type Conversion Operators
- 11.10 Convert Constructors
- 11.11 Aggregation and Composition
- 11.12 Inheritance
- 11.13 Protected members and Class Access
- 11.14 Constructors, Destructors and Inheritance
- 11.15 Overriding Base Class Functions